TripRescue

PRD v2.0 — Product Requirements Document
e Plano de Execução por Vertical Slices

Versão 2.0 • Base aprovada para iniciar desenvolvimento

Stack: React + TypeScript + Vite • FastAPI • Postgres (Vercel Marketplace)
Deploy: Vercel • CI: GitHub Actions • Repositório: monorepo

Documento substitui a v1.0. Incorpora decisões técnicas para acelerar o MVP.

Sumário

1. Como usar este documento
2. Visão do produto
3. Problema
4. Público-alvo
5. Objetivos e não objetivos
6. Escopo funcional por módulo
7. Requisitos funcionais
8. Requisitos não funcionais
9. Regras de domínio e segurança
10. Modelo de dados inicial
11. Arquitetura e decisões técnicas
12. Plano detalhado de execução (Vertical Slices)
13. Critérios globais de aceitação do MVP
14. Métricas iniciais
15. Organização recomendada do repositório
16. Rotina de trabalho e GitHub
17. Prompt para continuar em outra IA
18. Registro de decisões e questões abertas
19. Próxima ação concreta
20. Changelog em relação à v1.0

0. Como usar este documento

Este arquivo é a fonte única de contexto do projeto. Ao retomá-lo em outra conversa ou com outra IA:

1. Anexe este documento.
2. Compartilhe o estado real do repositório (`README.md` , árvore de arquivos, branch, últimos commits, erros atuais).
3. Peça para continuar **a partir da próxima fase não concluída**, sem presumir execução.
4. Atualize o checklist de execução e registre decisões em `docs/DECISIONS.md` .
5. Nunca copie chaves, tokens, senhas ou dados pessoais reais para prompts, commits ou issues.

> **Regra de continuidade:** este documento descreve o plano. Nenhuma fase é considerada executada até existir evidência no repositório (código, testes, commits).

1. Visão do produto

TripRescue é uma aplicação web responsiva que funciona como copiloto de viagem. Ajuda o viajante a:

- entender rapidamente o impacto de um imprevisto;
- saber qual ação tomar primeiro e por quê;
- distinguir fatos confirmados, inferências, estimativas e dados ausentes;
- acompanhar tarefas, evidências e histórico do incidente;
- priorizar segurança física acima de custo, tempo e conveniência.

O diferencial não é "mais um chatbot". É entregar um **plano priorizado, explicável e verificável** a partir do contexto da viagem e do incidente.

Princípios do produto

1. Segurança física e urgência têm prioridade sobre economia e conveniência.
2. O usuário mantém o controle de compras, reservas, mensagens e contato com autoridades.
3. O sistema distingue fatos, inferências, estimativas e dados ausentes.
4. Nunca inventar preços, disponibilidade, horários, contatos oficiais ou probabilidades.
5. Toda recomendação sensível deve indicar fonte, data ou contexto de verificação quando aplicável.
6. MVP funciona localmente e tem caminho documentado e funcional para a Vercel.
7. Fluxo vertical de ponta a ponta antes de expandir escopo.
8. Regras críticas são determinísticas e testáveis; IA é opcional e não substitui cálculo.

2. Problema

Durante uma viagem, atrasos, cancelamentos, conexões apertadas, perda de compromissos, fraude ou roubo exigem decisões rápidas. As informações ficam espalhadas entre bilhetes, e-mails, sites e regras de companhias. Sob pressão, o viajante perde tempo, gasta mais ou toma decisões inseguras.

TripRescue centraliza o contexto, explica impactos, organiza alternativas e transforma recomendações em ações acompanháveis.

3. Público-alvo

- Viajantes independentes, famílias e pequenos grupos.
- Pessoas com múltiplos trechos, conexões, transfers e compromissos.
- Pessoas que precisam organizar evidências e acompanhar solicitações após um incidente.
- Pessoas que precisam de orientação segura e fontes oficiais confirmadas em situações delicadas.

Necessidades principais

1. Entender rapidamente o que foi afetado.
2. Saber qual ação tomar primeiro e por quê.
3. Saber o que está confirmado e o que precisa de validação.
4. Guardar protocolos, comprovantes, notas e tarefas.
5. Não depender de IA como única fonte em emergências.

4. Objetivos e não objetivos

Objetivos do MVP

- Registrar manualmente viagem, trechos e compromissos.
- Detectar conflitos e riscos básicos por regras determinísticas testáveis.
- Registrar incidente e relacioná-lo aos itens afetados.
- Gerar plano de ação priorizado com motivo e estado.
- Exibir tarefas, notas e histórico do incidente.
- Manter código, testes e documentação no GitHub desde o início.
- Funcionar localmente e ter deploy funcional na Vercel.

Fora do escopo inicial (MVP)

- Apps nativos iOS/Android.
- Compra ou reserva automática de qualquer serviço.
- Envio automático de mensagens a autoridades ou companhias.
- Rastreamento de voos em tempo real.
- Probabilidades preditivas sem dados e método validados.

- Cobertura global, preços ao vivo ou disponibilidade sem fontes.
- Treinar modelo de IA próprio.
- **Módulo E (Alternativas e Custos)** — adiado para fase pós-MVP.
- **Upload de arquivos em Evidências** — adiado; MVP apenas com notas e metadados.

5. Escopo funcional por módulo

Módulo A — Viagens (MVP)

Criar, consultar, editar e excluir viagens com nome, origem/destino, datas, moeda, grupo e preferências. Trechos com origem, destino, transporte, horários e observações. Entrada manual.

Módulo B — Compromissos e conflitos (MVP)

Registrar hospedagens, transfers, passeios e compromissos com horário e localização textual. Detectar sobreposições, intervalos insuficientes e dependências por regras determinísticas. Cada alerta explica dados e regra usados.

Módulo C — Incidentes (MVP)

Registrar incidente com categoria, horário, descrição, gravidade contextual e elementos afetados. Mostrar impacto calculado com base em dados conhecidos e marcar o que é incerto.

Módulo D — Plano de ação (MVP)

Ações ordenadas por urgência, segurança, dependências e preferências. Cada ação contém:

- título e instrução concreta;
- prioridade e motivo;
- dados que fundamentam a recomendação;
- fonte ou caminho para confirmação quando relevante;
- estado: pendente, em andamento, concluída;
- aviso claro quando exige confirmação ou ação externa do usuário.

Módulo E — Alternativas e custos (Pós-MVP)

Comparação de opções com preço, duração, conveniência, acessibilidade e segurança. Adiado até existir fonte confiável de dados e termos de uso validados. No MVP, o plano de ação pode conter instruções do tipo "verifique opções de reacomodação com a companhia", sem inventar valores.

Módulo F — Evidências e acompanhamento (MVP simplificado)

Notas, protocolos e metadados em texto. Tarefas com estado e histórico. Rascunhos de mensagens sem envio automático. **Upload de arquivos fica fora do MVP** (requer armazenamento privado, validação, retenção e controle de acesso).

Módulo G — Segurança (MVP)

Risco à integridade física é prioridade máxima. Orientação breve, cautelosa e contextual. Canais oficiais apenas quando localização e fonte estiverem confirmadas. Em perigo imediato, orientar a priorizar local seguro e procurar serviços de emergência locais confirmados. Nunca alegar contato executado.

Módulo H — IA e fontes verificadas (Pós-MVP)

IA pode interpretar descrições, resumir e redigir rascunhos. Nunca é responsável por cálculo de horário, priorização crítica, avaliação de segurança ou dados de preço. Interface abstrata de provedor. Implementação só após fluxo determinístico estar sólido.

6. Requisitos funcionais

ID	Requisito	Módulo	Escopo
RF-01	Criar, consultar, editar e excluir viagem	A	MVP
RF-02	Registrar trechos com transporte, origem/destino e horários	A	MVP
RF-03	Registrar compromissos (hospedagem, transfer, passeio)	B	MVP
RF-04	Guardar preferências (prioridade, orçamento, grupo, acessibilidade)	A	MVP
RF-05	Detectar conflitos de tempo por regras explicáveis	B	MVP
RF-06	Registrar incidente com categoria, descrição e gravidade contextual	C	MVP
RF-07	Identificar trechos e compromissos potencialmente afetados	C	MVP
RF-08	Gerar plano de ação priorizado com motivo e estado	D	MVP
RF-09	Comparar alternativas com custo, duração e fonte	E	Pós-MVP
RF-10	Registrar notas e metadados de evidências	F	MVP
RF-11	Criar e atualizar tarefas com histórico básico	F	MVP
RF-12	Distinguir fatos, inferências, estimativas e dados ausentes	Transversal	MVP
RF-13	Priorizar segurança física e não alegar ações externas executadas	G	MVP
RF-14	Rascunhar mensagens sem envio automático	F	MVP

7. Requisitos não funcionais

- Interface responsiva e acessível, com estados de carregamento, erro e ausência de dados.
- Código modular, legível e testável; regras de negócio separadas de UI e IA.
- Segredos apenas em variáveis de ambiente; `.env.example` sem valores reais.
- Testes automatizados para regras de tempo, impacto, priorização e validação.
- Não registrar dados sensíveis desnecessários em logs.
- Dados fictícios em exemplos, testes e screenshots públicos.
- Autenticação e autorização antes de expor dados reais multiusuário.
- Build de produção reproduzível e instruções de execução local.
- CI automatizado (GitHub Actions) rodando testes a cada push/PR.
- Documentar limitações de hospedagem, banco e serviços externos.

8. Regras de domínio e segurança

1. Segurança imediata antes de custo, velocidade e conveniência.
2. Não gerar porcentagens de risco sem dados e método validados.
3. Conflitos temporais calculados por código determinístico, não por IA.
4. Dado desconhecido permanece desconhecido — nunca completar com invenção.
5. Toda estimativa de preço é identificada com premissas (quando existir).
6. Não apresentar informação publicada como cotação garantida ou disponibilidade em tempo real.
7. Não alegar que reserva, compra, denúncia, ligação ou mensagem foi executada sem integração real e confirmação.
8. Recomendações de emergência usam fontes oficiais confirmadas e não atrasam segurança imediata.
9. Fonte externa indisponível = informar limitação e oferecer caminho manual seguro.
10. Minimizar coleta, retenção e exposição de dados pessoais.

9. Modelo de dados inicial

> Evolui a partir dos fluxos reais. Não criar tabelas sem necessidade demonstrada.

- **Trip:** id, título, origem, destino, datas, moeda, preferências essenciais.
- **TripSegment:** trip_id, tipo de transporte, origem/destino, horários planejados, observações.
- **Commitment:** tipo, nome, horário, localização textual, trip_id/segment_id.
- **Incident:** categoria, descrição, momento, gravidade contextual, estado.
- **IncidentAffectedItem:** incident_id, referência a segment/commitment, impacto calculado.
- **ActionTask:** título, instrução, prioridade, motivo, estado, prazo opcional, incident_id.
- **EvidenceRecord:** tipo, descrição, data, conteúdo textual (sem upload no MVP).

- **Alternative:** (Pós-MVP) tipo, custo, moeda, duração, fonte, momento da consulta, estado.
- **User/Auth:** (Pós-MVP) introduzir quando a estratégia de autenticação for definida.

10. Arquitetura e decisões técnicas

Decidido (v2.0)

Área	Decisão	Justificativa
Editor	VS Code	Padrão do usuário
Versionamento	Git + GitHub desde o dia 1	Rastreabilidade
Frontend	React + TypeScript + Vite	Stack leve e moderna
Backend	Python + FastAPI	Suporte nativo a Vercel Functions (zero-config)
Deploy	Vercel (frontend + backend)	Fluid Compute suporta FastAPI; sem serviço separado no MVP
Banco em produção	Vercel Marketplace (Neon ou Supabase Postgres)	Provisionamento com 1 clique, credenciais automáticas
Banco local	SQLite + SQLAlchemy	Zero instalação, desenvolvimento rápido, migração fácil
Gerenciador Python	`uv`	Instalação e resolução drasticamente mais rápidas
Gerenciador Node	`pnpm`	Rápido e eficiente em disco
Testes backend	`pytest`	Padrão do ecossistema
Testes frontend	`vitest`	Integrado ao Vite
CI/CD	GitHub Actions + Vercel (auto-deploy na `main`)	Feedback contínuo
Autenticação	Não implementada no MVP (single-user)	Adiada até multiusuário
IA	Provedor não escolhido; fase posterior	Fluxo determinístico primeiro

Ainda em aberto

- Monorepo vs repositórios separados (recomendação: **monorepo** com `frontend/` e `backend/`).
- Biblioteca de componentes do frontend (avaliar `shadcn/ui` ou componentes próprios enxutos).
- Estratégia de gerenciamento de estado (provavelmente `React Query` + estado local).
- Provedor de IA, embeddings e RAG (pós-MVP).
- APIs externas de voos e preços (pós-MVP).

- Política de retenção e exclusão de dados (pós-MVP).

11. Plano detalhado de execução (Vertical Slices)

> Cada fase termina com resultado executável, testes relevantes e commit no GitHub. Não avançar se algo falhar sem registrar o erro e resolvê-lo. **Cada slice deve funcionar de ponta a ponta (frontend + backend + banco).**

Fase 0 — Preparação e repositório

Objetivo: base versionada, limpa e reproduzível.

1. Verificar instalações: Git, Node.js, `pnpm`, Python, `uv`.
2. Criar repositório no GitHub (monorepo) e clonar no VS Code.
3. Criar `README.md` com visão, stack e status real.
4. Criar `.gitignore` (Node, Python, `uv`, caches, builds, `.env*` exceto `.env.example`).
5. Criar `.env.example` sem valores secretos.
6. Criar `docs/PRD.md` (este documento) e `docs/DECISIONS.md`.
7. Convenção de commits: `feat:`, `fix:`, `docs:`, `test:`, `chore:`, `refactor:`.
8. Confirmar ausência de segredos/dados reais.
9. Primeiro commit e push.

Aceitação: repositório clonado, documentação presente, primeiro commit enviado, `git status` limpo.

Fase 1 — Esqueleto executável local + deploy inicial

Objetivo: provar comunicação frontend ↔ backend local e na Vercel.

1. `frontend/` com React + TypeScript + Vite (`npm create vite@latest`).
2. `backend/` com FastAPI + `uv` (virtualenv gerenciado por `uv`).
3. Endpoint `GET /health` retornando `{ "status": "ok" }`.
4. Tela inicial que chama `/health` e exibe "API conectada" ou erro legível.
5. URL da API via `VITE_API_URL` no frontend.
6. Scripts separados para rodar frontend e backend; documentar no `README`.
7. Testes mínimos: `pytest` para `/health`, `vitest` para o componente.
8. Configurar deploy Vercel (preview). Validar `/health` público.
9. GitHub Actions: rodar testes em push/PR.

Aceitação: ambos iniciam localmente; interface consome API; deploy de preview acessível; CI verde.

Fase 2 — Slice vertical: Viagem

Objetivo: primeira funcionalidade real de ponta a ponta.

1. Definir formulário mínimo de viagem (nome, origem/destino, datas, moeda, grupo).
2. Validar campos obrigatórios e datas no frontend e backend.
3. Modelos SQLAlchemy + schemas Pydantic.
4. Endpoints: `POST /trips`, `GET /trips`, `GET /trips/{id}`, `PATCH /trips/{id}`.
5. Banco local SQLite; preparar `DATABASE_URL` para Postgres via env.
6. Telas: listagem e detalhe.
7. Testes: validação, persistência, casos de erro.
8. Atualizar `README` e `DECISIONS`.

Aceitação: usuário cria, lista e edita viagens; dados sobrevivem ao restart; testes verdes; deploy Vercel atualizado.

Fase 3 — Slice vertical: Trechos e Compromissos

Objetivo: representar itinerário completo.

1. CRUD de trechos (origem, destino, transporte, horários, observações).
2. CRUD de compromissos (tipo, nome, horário, local textual).
3. Validação de ordem e coerência temporal — **sem presumir fusos não informados**.
4. Linha do tempo do itinerário na UI.
5. Testes: horários ausentes, ordem inválida, sobreposição, limites de data.

Aceitação: usuário registra 2+ trechos e 1 compromisso; timeline exibida; erros de validação claros.

Fase 4 — Motor determinístico de conflitos

Objetivo: regras explicáveis de detecção de risco temporal.

1. Definir regras explícitas (sobreposição, intervalo mínimo configurável, dependências).
2. Funções de domínio puras e testáveis em `backend/domain/rules/`.
3. Cada alerta referencia os dados que o causaram (IDs e campos).
4. Severidade qualitativa (ex.: `low`, `medium`, `high`) com motivo — **sem percentuais**.
5. Testes cobrindo: caso normal, fronteira de horário, dados incompletos, fuso não informado.
6. Exemplos de demonstração com dados fictícios.
7. Exibir alertas na UI com explicação.

Aceitação: ≥ 1 conflito detectado e explicado; testes críticos passam; usuário entende e corrige dados.

Fase 5 — Slice vertical: Incidente e Plano de Ação

Objetivo: coração do produto — transformar problema em plano.

1. Formulário de incidente: categoria (atraso, cancelamento, conexão em risco, outro), descrição, momento.
2. Relacionar incidente a trechos/compromissos (afetados explícitos).
3. Regras de prioridade transparentes em `backend/domain/prioritization/`.
4. Gerar `ActionTask` com título, instrução, motivo, estado.
5. UI: marcar tarefas como pendente / em andamento / concluída.
6. Indicadores claros do que exige **confirmação externa pelo usuário**.
7. Testes: incidente com informação parcial, sem alternativa conhecida, múltiplos afetados.

Aceitação: usuário registra incidente, vê impactos explicados, acompanha tarefas; sistema não alega execução externa.

Fase 6 — Evidências (texto) e Acompanhamento

Objetivo: histórico organizado, sem upload.

1. Notas e protocolos em texto associados ao incidente.
2. Histórico de atualização de tarefas (timestamps + estado anterior/novo).
3. Rascunhos de mensagens (texto gerado localmente, sem envio).
4. Nenhum conteúdo sensível em logs.
5. Testes de criação, atualização e listagem.

Aceitação: usuário consulta histórico e atualiza tarefas; nenhuma mensagem apresentada como enviada; sem uploads.

Fase 7 — Fluxo de Segurança

Objetivo: experiência cautelosa para risco físico.

1. Definir categorias e regras de escalonamento qualitativo.
2. Segurança física acima de custo/conveniência no plano de ação.
3. Localização informada explicitamente pelo usuário (sem inferência).
4. Fontes oficiais verificadas apenas quando localização e fonte confirmadas.
5. Exibir fonte e data de revisão quando aplicável.
6. Mensagens curtas, sem incentivo a confronto, priorizando segurança.
7. Testes: fonte ausente, localização incerta, perigo imediato.
8. **Proibido** implementar alegação de contato com polícia/consulado/emergência.

Aceitação: fluxo não inventa contatos; limitações claras; prioriza segurança física.

Fase 8 — Qualidade, Privacidade e Segurança

Objetivo: preparar MVP para teste externo.

1. Validar inputs no frontend e backend.

2. Revisar CORS, headers e proteção de endpoints.
3. Verificar dependências, segredos, logs e mensagens de erro.
4. Testes de integração dos fluxos principais (viagem → incidente → plano).
5. Testar responsividade e acessibilidade básica (contraste, foco, labels).
6. Documentar limitações conhecidas.
7. Dados fictícios na demonstração pública.
8. Se multi-usuário surgir, **antes** de dados reais: implementar autenticação.

Aceitação: fluxos essenciais com testes; nenhum segredo versionado; riscos documentados.

Fase 9 — Deploy de produção e validação

Objetivo: publicação estável na Vercel.

1. Provisionar Postgres via Vercel Marketplace (Neon ou Supabase).
2. Migrar de SQLite local para Postgres em produção (env `DATABASE_URL`).
3. Rodar migrações (definir ferramenta: `alembic` recomendado).
4. Configurar variáveis de produção (sem segredos no frontend).
5. Configurar CORS com domínio de produção.
6. Rodar build e testes em CI antes do deploy.
7. Deploy de preview → validar → promover a produção.
8. Testar `/health`, criação de viagem, registro de incidente, plano de ação.
9. Documentar rollback e rotação de variáveis.

Aceitação: URL pública funcional; frontend alcança API; dados persistem; nenhum segredo exposto; rollback documentado.

Fase 10 — Demonstração e documentação final

Objetivo: projeto demonstrável e reproduzível.

1. `README` com problema, funcionalidades reais, stack, arquitetura, screenshots.
2. Instruções reproduzíveis de instalação e execução local.
3. Registrar o que funciona, o que é simulado e o que está planejado.
4. Cenário de demonstração com dados fictícios.
5. Lista de limitações e próximos passos.
6. Revisar histórico de commits (remover artefatos/segredos).
7. Coletar feedback e registrar melhorias priorizadas em `docs/DECISIONS.md`.

Aceitação: outra pessoa entende, executa e testa o fluxo central sem o autor.

Fase 11 — Pós-MVP (backlog)

Só iniciar após MVP validado:

- Módulo E (Alternativas e Custos) com fonte real.
- Upload de evidências com armazenamento privado e retenção.
- Autenticação multiusuário.
- Integração de IA com interface abstrata de provedor.
- APIs externas de voos/preços com verificação de termos.

12. Critérios globais de aceitação do MVP

- Usuário cria viagem com ≥ 2 trechos e ≥ 1 compromisso.
- Sistema identifica ≥ 1 conflito de tempo e explica a regra usada.
- Usuário registra incidente e visualiza itens afetados.
- Plano de ação é priorizado, com motivo e tarefas acompanháveis.
- Dados ausentes não são inventados.
- Histórico básico consultável e tarefas atualizáveis.
- Fluxo de segurança não inventa contatos nem alega ações externas.
- Testes principais passam em CI.
- Projeto funciona localmente conforme `README`.
- Publicação funcional na Vercel com `health` e fluxo central validados.
- Nenhuma credencial versionada.

13. Métricas iniciais

Sem metas numéricas antes de linha de base. Medir:

- sucesso dos testes de conflito;
- tempo para identificar próxima ação em cenários simulados;
- proporção de recomendações com origem e limitações explícitas;
- sucesso dos fluxos essenciais em teste manual;
- feedback de clareza, confiança e utilidade;
- erros de execução local e de deploy.

14. Organização recomendada do repositório (monorepo)

```
TripRescue/
├── .github/
│   └── workflows/
│       ├── backend-ci.yml
│       └── frontend-ci.yml
├── .gitignore
├── .env.example
├── README.md
├── docs/
│   ├── PRD.md
│   ├── DECISIONS.md
│   └── DEMO.md
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   ├── tsconfig.json
│   ├── index.html
│   └── src/
│       ├── main.tsx
│       ├── App.tsx
│       ├── api/
│       ├── components/
│       ├── pages/
│       └── __tests__/
├── backend/
│   ├── pyproject.toml
│   ├── uv.lock
│   ├── alembic.ini
│   ├── alembic/
│   └── app/
│       ├── main.py
│       ├── api/
│       ├── domain/
│       │   └── rules/
│       ├── models/
│       ├── schemas/
│       └── tests/
└── vercel.json
```

15. Rotina de trabalho e GitHub

Para cada passo:

1. Ler a tarefa e definir resultado esperado.
2. Confirmar estado real do repositório.
3. Fazer a menor alteração que entregue valor verificável.
4. Rodar testes e build relevantes.
5. Corrigir erros antes de continuar.
6. Atualizar documentação (`README` , `DECISIONS`).

7. Commit com mensagem clara (`feat:` , `fix:` , etc.).
8. Push e confirmar estado no GitHub.
9. Registrar decisões e pendências.

> **Nunca commitar** `.env` , tokens, senhas, dados de usuário ou arquivos locais. Se um segredo for exposto, **revogar/rotacionar** — remover do código não basta.

16. Prompt para continuar em outra IA

> Você está ajudando a desenvolver o TripRescue. Leia integralmente este PRD (v2.0). Ele define visão, escopo, decisões técnicas, requisitos e fases. O documento é o plano, não prova de execução.
>
> O usuário desenvolve localmente no VS Code, versiona no GitHub desde o início, e o MVP deve funcionar localmente e na Vercel. Stack: React + TS + Vite (frontend), FastAPI (backend), SQLite local + Postgres via Vercel Marketplace (produção), `uv` e `pnpm` , CI via GitHub Actions. Autenticação e IA são pós-MVP.
>
> Antes de modificar arquivos, confirme o estado real do repositório, branch, arquivos e últimos commits. Identifique a próxima fase não concluída. Trabalhe em **vertical slices** — cada funcionalidade de ponta a ponta (frontend + backend + banco). Não avance após teste falhar sem tratar ou registrar o problema.
>
> Nunca invente preços, fontes, contatos, probabilidades ou funcionalidades. Segurança física vem antes de custo e conveniência. Não alegue execuções externas. Não exponha segredos. Ao concluir cada etapa, informe arquivos alterados, comandos executados, resultados reais de testes, commit recomendado e próxima etapa.

17. Registro de decisões e questões abertas

Decisões tomadas (v2.0)

- Nome: TripRescue.
- Aplicação web responsiva.
- Stack: React + TS + Vite / FastAPI.
- Deploy: Vercel (frontend + backend via Functions).
- Banco produção: Postgres via Vercel Marketplace (Neon/Supabase).
- Banco local: SQLite + SQLAlchemy; migrações com Alembic.
- Gerenciadores: `uv` (Python) e `pnpm` (Node).
- CI: GitHub Actions desde o dia 1.
- Monorepo com `frontend/` e `backend/` .
- Autenticação e IA: pós-MVP.

- Módulos E e uploads: pós-MVP.
- Prioridade em vertical slices.

Questões abertas

- Biblioteca de componentes (avaliar `shadcn/ui` vs. componentes próprios).
- Gerenciamento de estado (avaliar React Query).
- Escolha entre Neon e Supabase para Postgres de produção (decidir na Fase 9 com base em limites e custo).
- Provedor de IA e RAG (pós-MVP).
- APIs externas de voos/preços (pós-MVP).
- Estratégia de localização e fontes oficiais por país (Fase 7).
- Política de retenção e exclusão de dados (pós-MVP).

18. Próxima ação concreta

Iniciar a Fase 0. Antes de escrever código:

1. Verificar versões: `git --version`, `node --version`, `pnpm --version`, `python --version`, `uv --version`.
2. Criar repositório no GitHub e clonar localmente.
3. Criar `README.md`, `.gitignore`, `.env.example`, `docs/PRD.md` (este documento), `docs/DECISIONS.md`.
4. Primeiro commit e push.

Nenhuma integração externa ou funcionalidade avançada antes do esqueleto local estar funcionando e o deploy de preview na Vercel responder.

19. Changelog em relação à v1.0

Adicionado:

- Módulo D explicitado (Plano de Ação).
- Vercel como ambiente para FastAPI (zero-config) desde o início.
- Vercel Marketplace para Postgres em produção.
- SQLite + SQLAlchemy para desenvolvimento local; Alembic para migrações.
- `uv` e `pnpm` como gerenciadores padrão.
- GitHub Actions desde o dia 1.
- Monorepo com estrutura explícita.
- Execução em **vertical slices** (cada fase entrega funcionalidade de ponta a ponta).
- Fase 9 separada para deploy de produção e validação.

- Fase 11 com backlog pós-MVP.

Movido para pós-MVP:

- Módulo E (Alternativas e Custos).
- Upload de arquivos em Evidências.
- Autenticação multiusuário.
- IA e RAG.

Corrigido:

- Numeração de módulos (A, B, C, D, E, F, G, H).
- Árvore de repositório legível.
- Prompt de continuidade atualizado com stack v2.0.

TripRescue — PRD v2.0 • Documento vivo. Se algo conflitar com o estado real do repositório, o código e `docs/DECISIONS.md` prevalecem.